

JAVA PROGRAMMING PRACTICAL RECORD

Bank Functionality – Dynamic Polymorphism

Experiment	Bank Functionality using Dynamic Polymorphism
Concept	Inheritance and Runtime Polymorphism
Language	Java

1. Aim

To write a Java program to implement bank functionality according to RBI rules and demonstrate dynamic polymorphism using RBI as the base class and SBI, ICICI and PNB as derived classes.

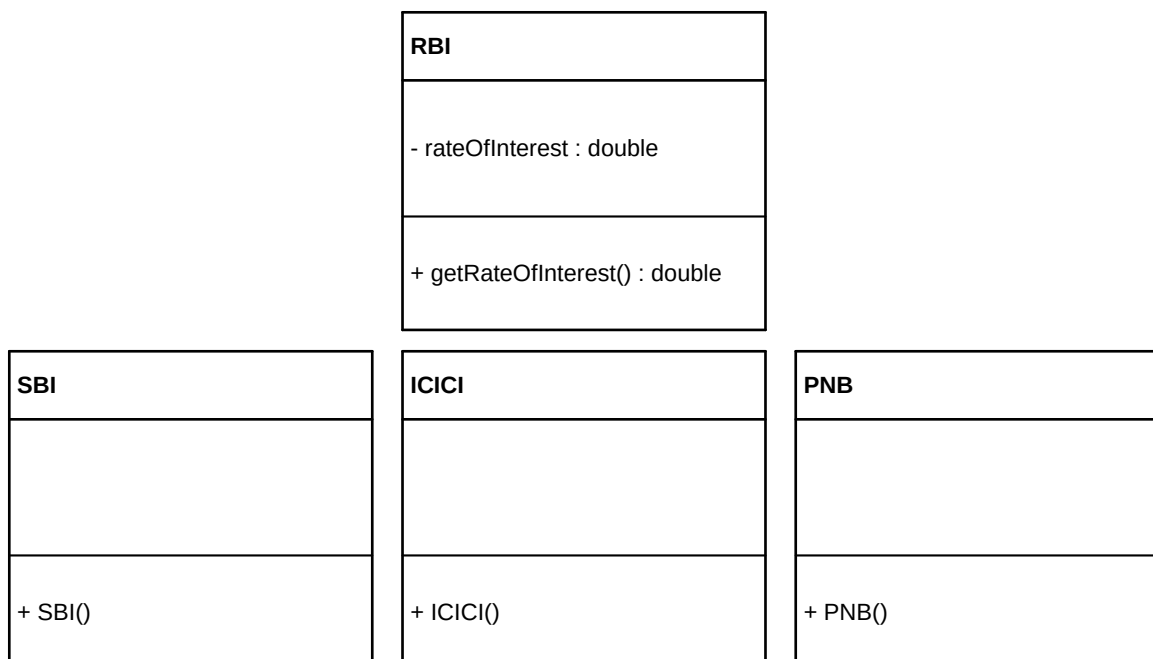
2. Problem Statement

RBI specifies minimum banking rules such as interest rates, while individual banks may provide rates equal to or greater than the RBI rate. Create Customer, Account, RBI and derived bank classes and display the applicable rate of interest based on the bank selected by the user.

3. Algorithm

1. Create a Customer class with customer name and customer ID.
2. Create an Account class with account number, balance and Customer object.
3. Create RBI as the base class with a default interest rate of 4%.
4. Create SBI, ICICI and PNB classes by extending RBI.
5. Override the interest rate in each derived bank using its constructor.
6. Create an RBI reference variable in the main method.
7. Create an object according to the bank name entered by the user.
8. Use the RBI reference to call getRateOfInterest().
9. Display the applicable rate of interest.

4. UML Class Diagram



Inheritance: SBI, ICICI and PNB extend RBI

5. Java Program

```
import java.util.Scanner;

class Customer {
    String name;
    int customerId;

    Customer(String name, int customerId) {
        this.name = name;
        this.customerId = customerId;
    }

    void displayCustomer() {
        System.out.println("Customer Name: " + name);
        System.out.println("Customer ID: " + customerId);
    }
}

class Account {
    int accountNumber;
    double balance;
    Customer customer;

    Account(int accountNumber, double balance, Customer customer) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.customer = customer;
    }

    void displayAccount() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }
}

class RBI {
    double rateOfInterest = 4.0;

    double getRateOfInterest() {
        return rateOfInterest;
    }
}

class SBI extends RBI {
    SBI() {
        rateOfInterest = 7.0;
    }
}

class ICICI extends RBI {
    ICICI() {
        rateOfInterest = 6.5;
    }
}

class PNB extends RBI {
    PNB() {
        rateOfInterest = 6.0;
    }
}

public class BankDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the Bank name to find the rate of Interest : ");
        String bankName = sc.nextLine();

        RBI bank;

        if (bankName.equalsIgnoreCase("RBI")) {
            bank = new RBI();
        } else if (bankName.equalsIgnoreCase("SBI")) {
```

```

        bank = new SBI();
    } else if (bankName.equalsIgnoreCase("ICICI")) {
        bank = new ICICI();
    } else if (bankName.equalsIgnoreCase("PNB")) {
        bank = new PNB();
    } else {
        System.out.println("Invalid Bank Name");
        sc.close();
        return;
    }

    System.out.println("RBI rate of interest is : "
        + bank.getRateOfInterest() + "%");

    sc.close();
}
}

```

6. Sample Output – Test Case 1

```

Enter the Bank name to find the rate of Interest : RBI
RBI rate of interest is : 4.0%

```

7. Sample Output – Test Case 2

```

Enter the Bank name to find the rate of Interest : SBI
RBI rate of interest is : 7.0%

```

8. Dynamic Polymorphism

Dynamic polymorphism is demonstrated by the RBI reference variable:

```

RBI bank;
bank = new RBI();
bank = new SBI();
bank = new ICICI();
bank = new PNB();

```

The reference type is RBI, but the actual object may be RBI, SBI, ICICI or PNB. At runtime, the appropriate object-specific rate is obtained through the same RBI reference. This demonstrates runtime/dynamic polymorphism.

9. Result

Thus, the Java program successfully implements the RBI and bank functionality and demonstrates dynamic polymorphism using inheritance and an RBI base-class reference.